

Introduction to Artificial Intelligence

Lab 6: Reinforcement Learning - Group 24, Variant 4

By: Sai Koushik Neriyanuri and Gülten Sezer

Introduction

This report documents the development and evaluation of a Q-Learning based reinforcement learning agent designed to solve the Taxi-v3 problem, a well-defined task from the Gymnasium library. The objective of the Taxi-v3 environment is to pick up a passenger at one of several designated locations, navigate to the correct destination, and drop the passenger off, all while minimizing the travel time and incorrect actions.

Task at Hand

The task involves controlling a taxi in a grid environment where the taxi needs to pick up a passenger at a specified location and drop them off at another, with the environment providing a reward structure that penalizes unnecessary movements and rewards successful pickups and drop-offs. The goal is to maximize the total reward by developing an efficient driving strategy.

Code Explanation

The implemented code uses Python with the Gymnasium library for the environment simulation. Key components of the code include:

- **Environment Setup:** The Taxi-v3 environment is initialized, which defines the state and action space.
- **Q-Table Initialization:** A table of Q-values (state-action pairs) is initialized to either all zeros or small random values to test different initialization strategies.
- **Learning Parameters:** Hyperparameters such as the learning rate (**alpha**), discount factor (**gamma**), and parameters for the epsilon-greedy policy (**epsilon**, **epsilon_decay**, **epsilon_min**) are defined to control the learning dynamics.
- **Learning Loop:** The agent iterates over episodes, within each it chooses actions based on the current policy (epsilon-greedy), observes the consequences (next state and reward), and updates the Q-values based on the observed transitions.

- **Visualization and Metrics:** The performance of the agent is tracked through the total reward per episode and the number of steps per episode, and the convergence rate, both plotted to observe learning progress and efficiency.

Visualization using Graphs:

Total Rewards per Episode Graph: This graph shows how the total reward evolves over episodes. Initially, the agent receives more negative rewards, indicating frequent wrong actions or inefficiencies. Over time, as the agent learns, the total rewards should increase (or become less negative), indicating more successful episodes.

Steps per Episode Graph: Depicts the number of steps the agent takes to complete each episode. A decrease in steps over time suggests that the agent is learning to solve the task more efficiently, potentially finding shorter or more direct routes to the passenger and destination.

Average Q-value Change per Episode Graph: This graph tracks the average change in the Q-values across all state-action pairs per episode, offering insight into the convergence of the agent's learning. A sharp initial decline followed by a levelling off indicates rapid learning and stabilization of the policy. The decreasing trend shows that the agent is making fewer adjustments to its Q-values over time, signifying a settling on an optimal strategy.

Experimentations with different values of hyperparameters:

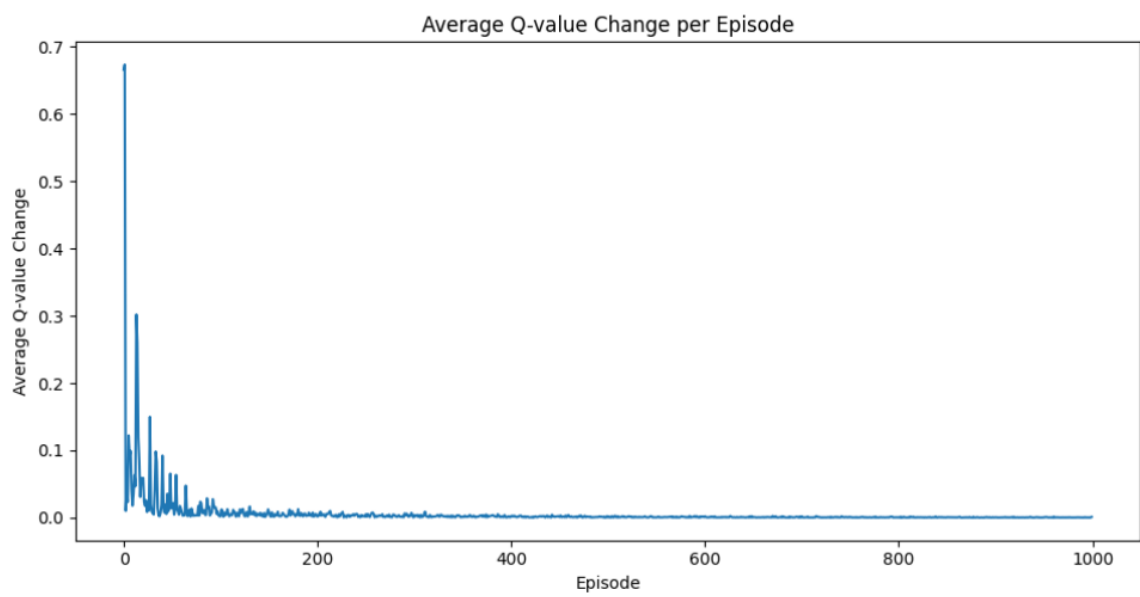
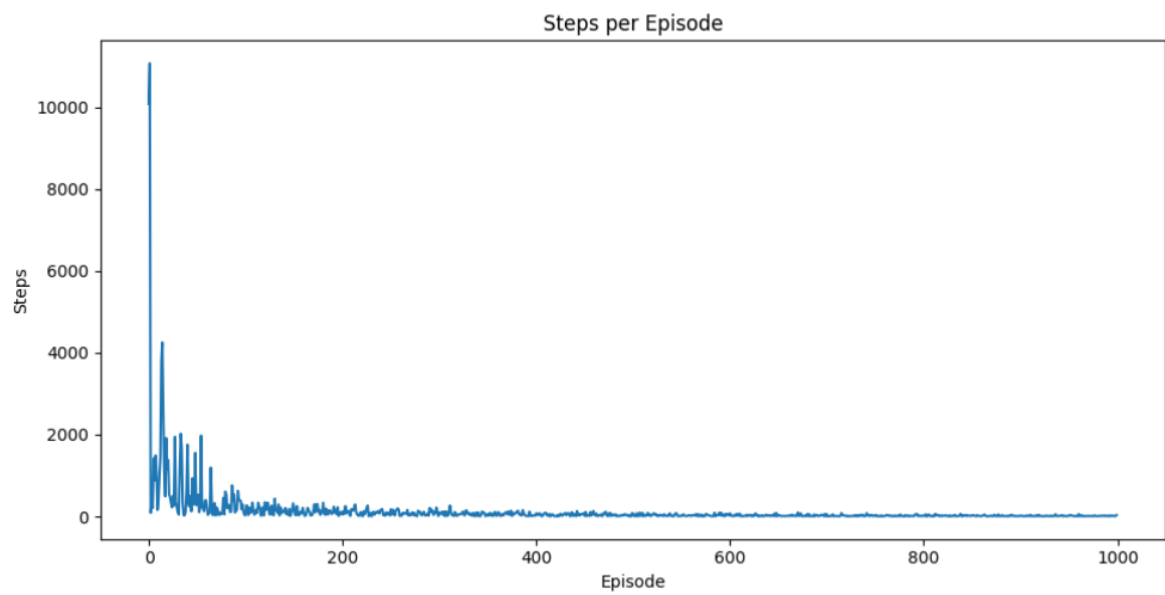
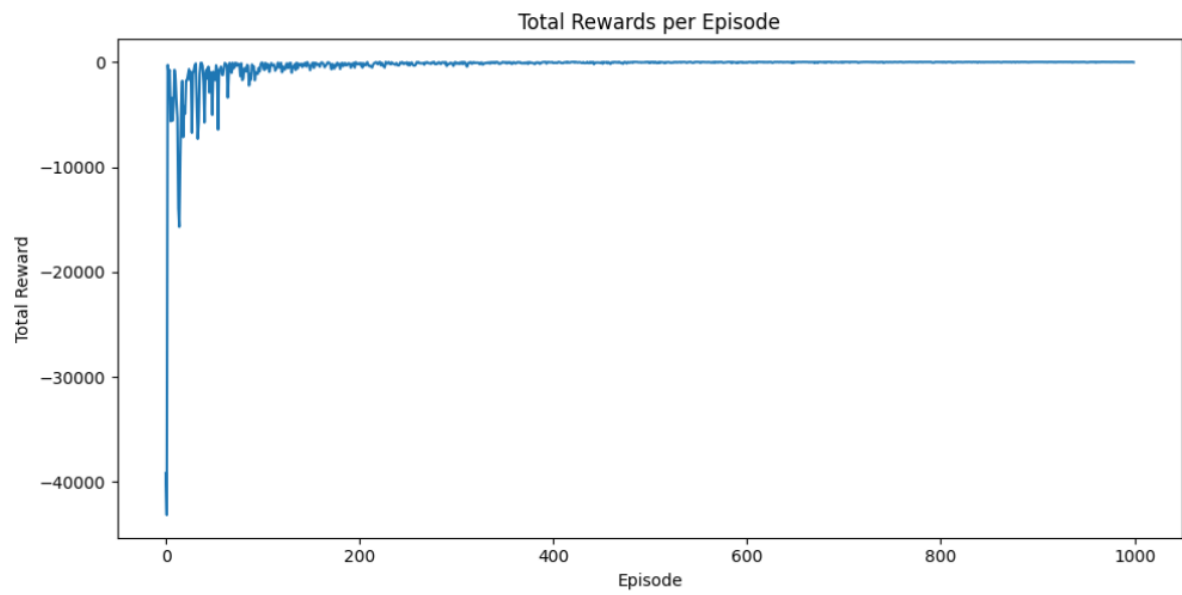
1. Q-table Initialization

Experiment 1: Zero Initialization

Initialize the Q-table with zeros. This is the baseline setup to compare other initializations against.

```
Q = np.zeros((n_states, n_actions))
```

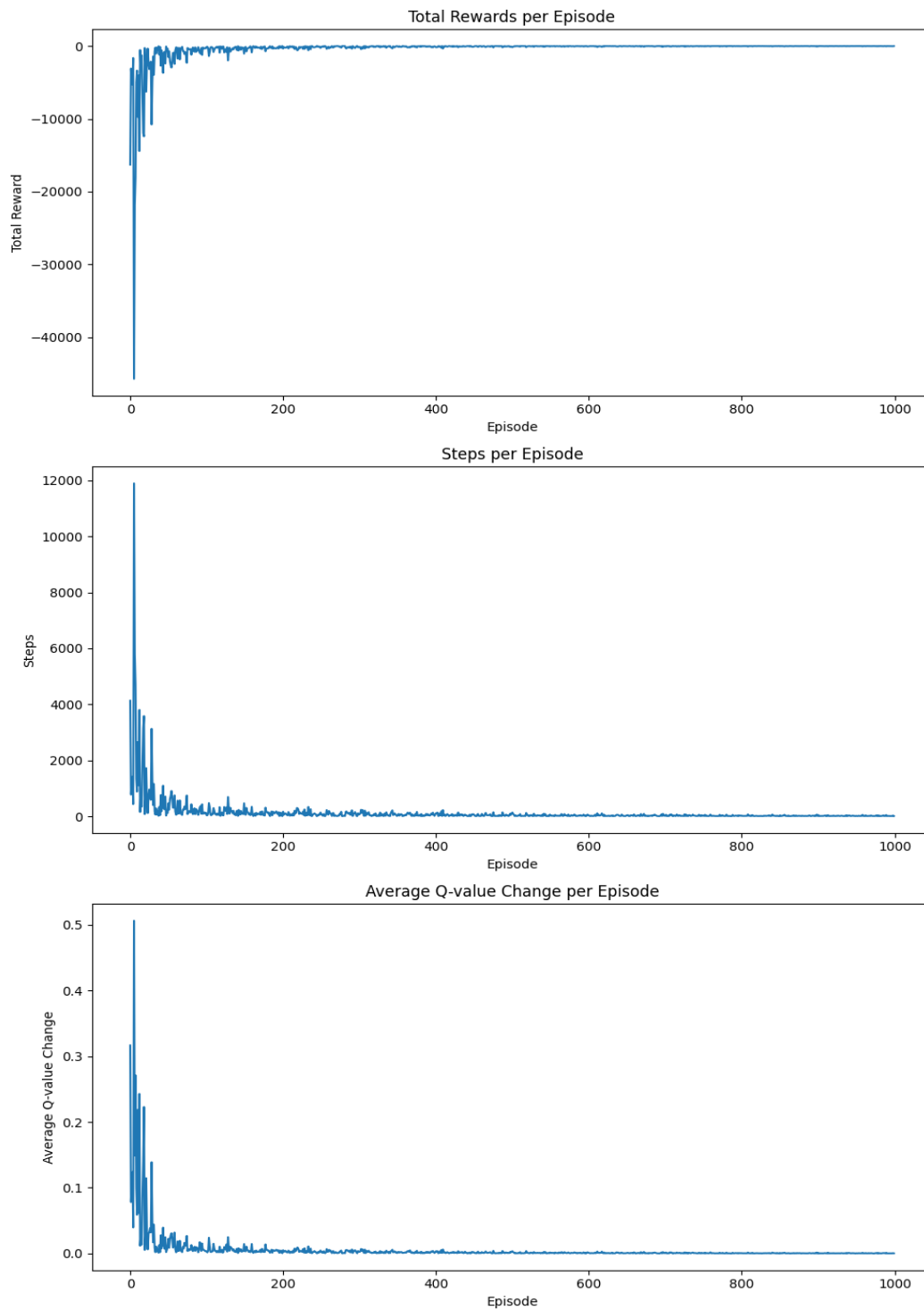
- **Hyperparameters:**
 - $\alpha = 0.1$
 - $\gamma = 0.95$
 - $\epsilon = 1.0$
 - $\epsilon_{\min} = 0.01$
 - $\epsilon_{\text{decay}} = 0.995$



Experiment 2: Random Initialization

- Initialize the Q-table with small random values to encourage exploration

```
Q = np.random.uniform(low=-2, high=0, size=(n_states, n_actions))
```



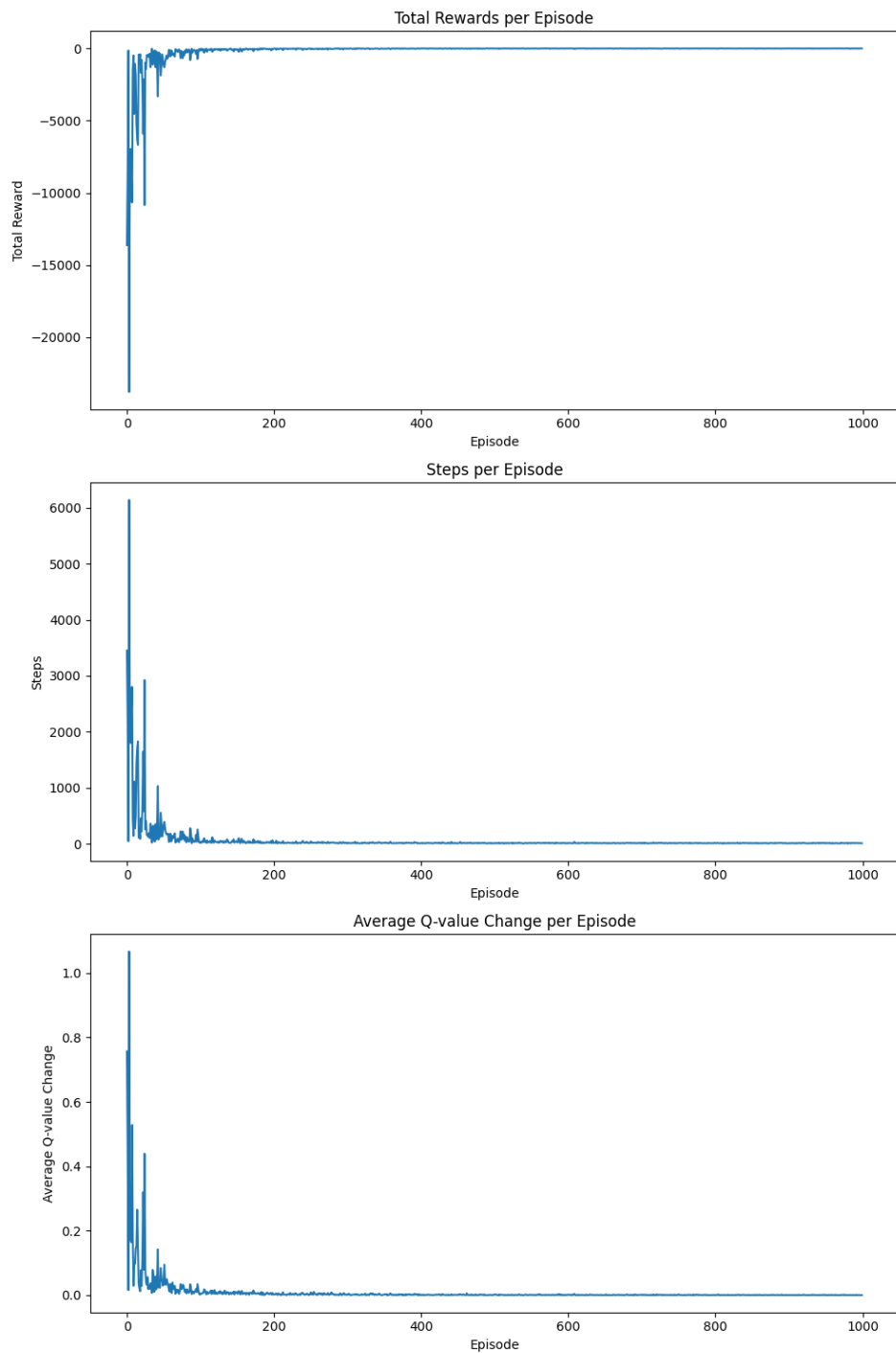
2. Learning Rate (Alpha)

Experiment 3: High Learning Rate

- Use the zero initialization for the Q-table.

Hyperparameters:

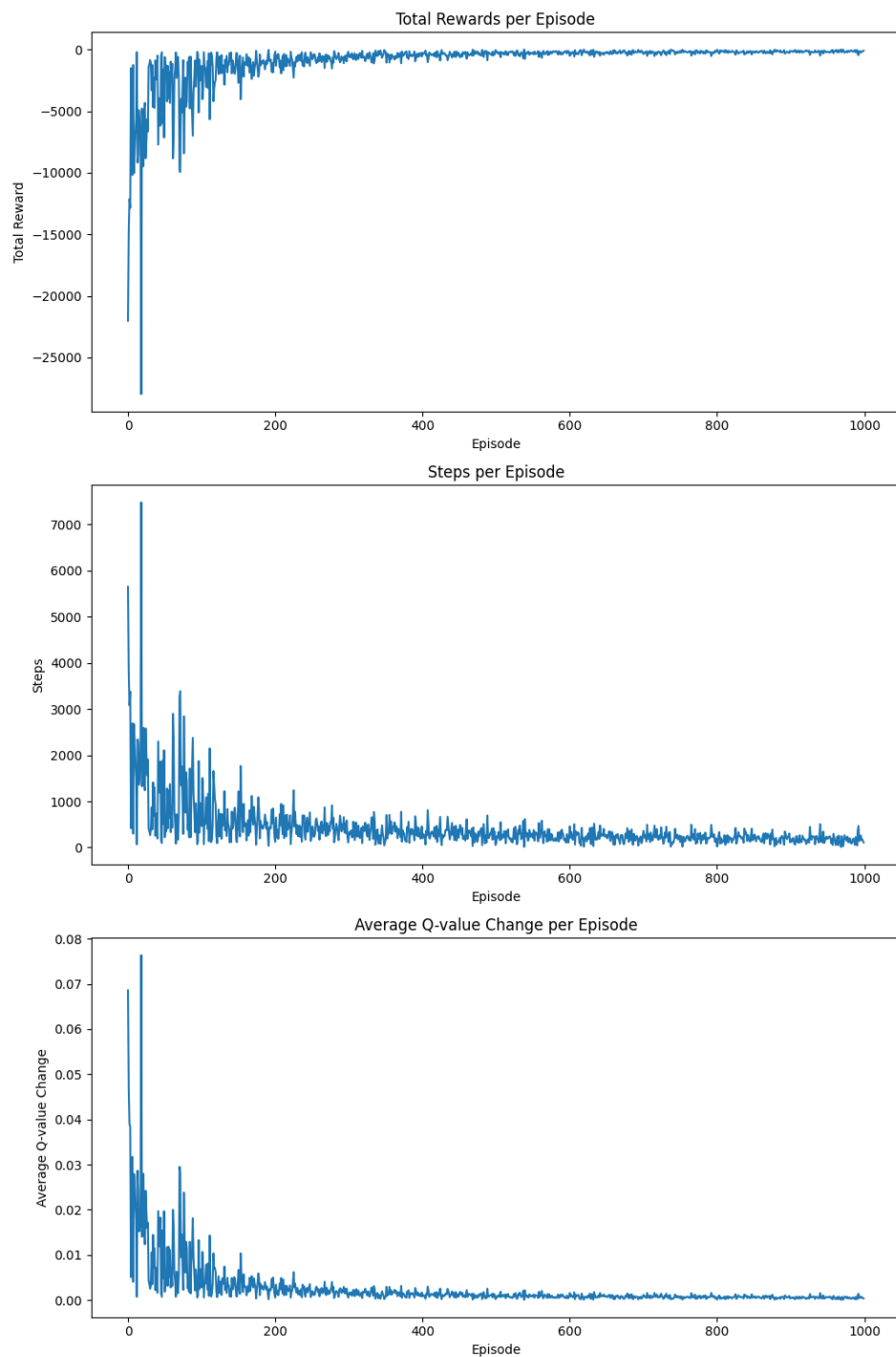
- $\alpha = 0.5$ - Higher learning rate



Experiment 4: Low Learning Rate

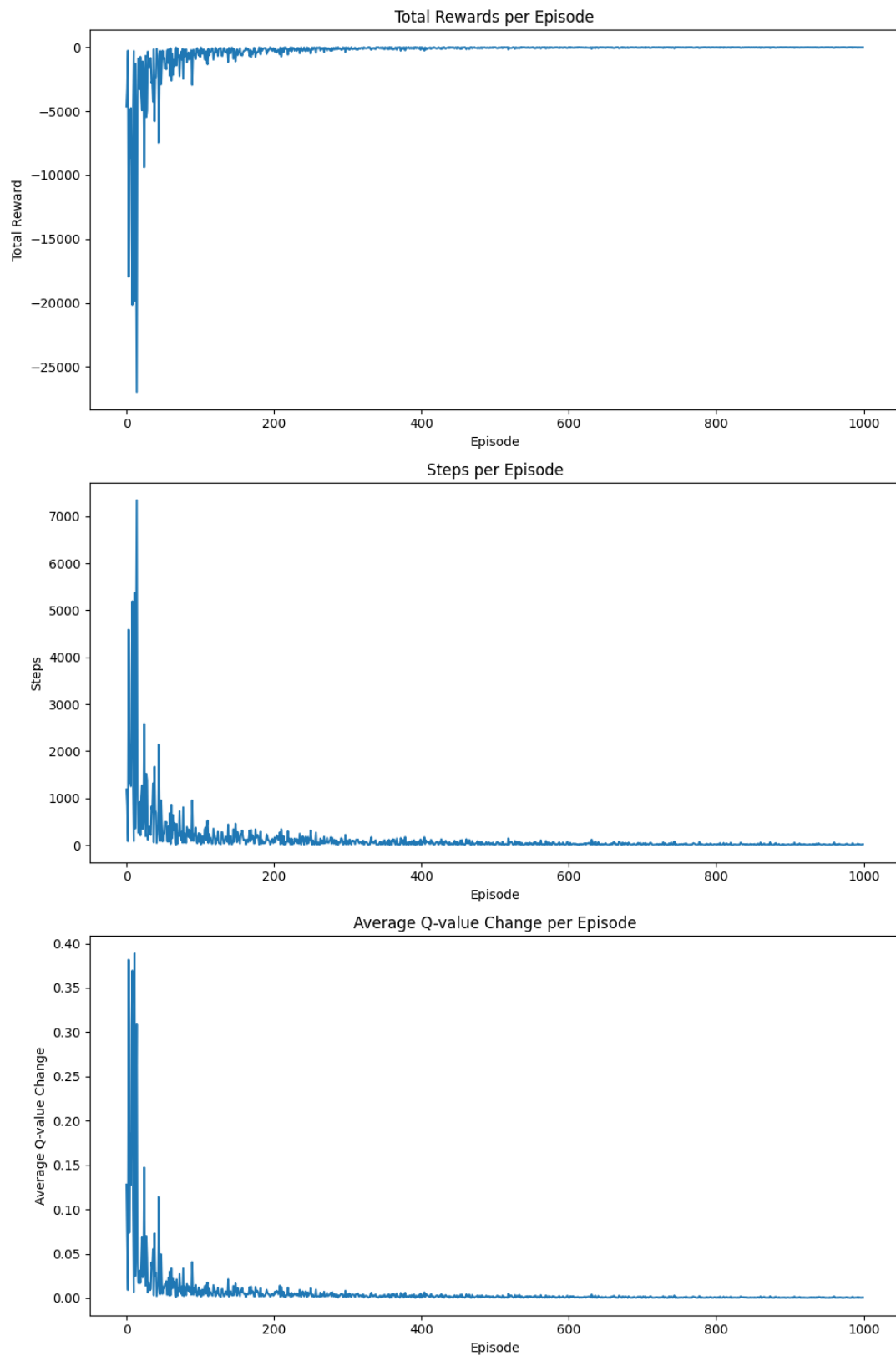
Hyperparameters:

- $\alpha = 0.01$ - Lower learning rate



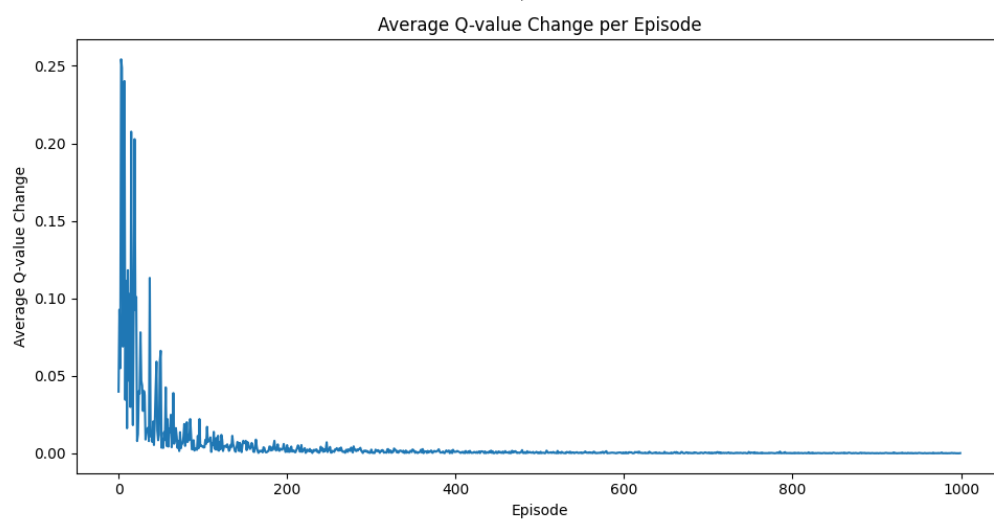
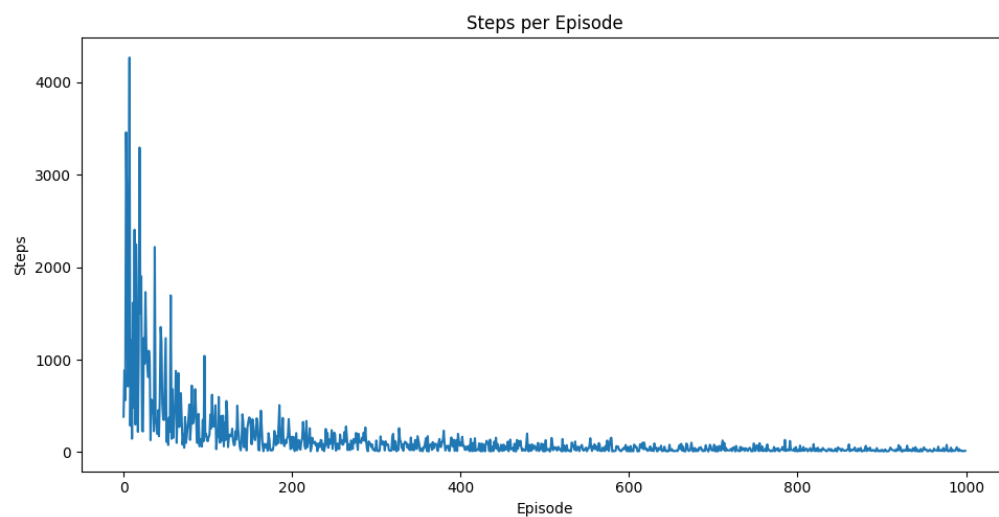
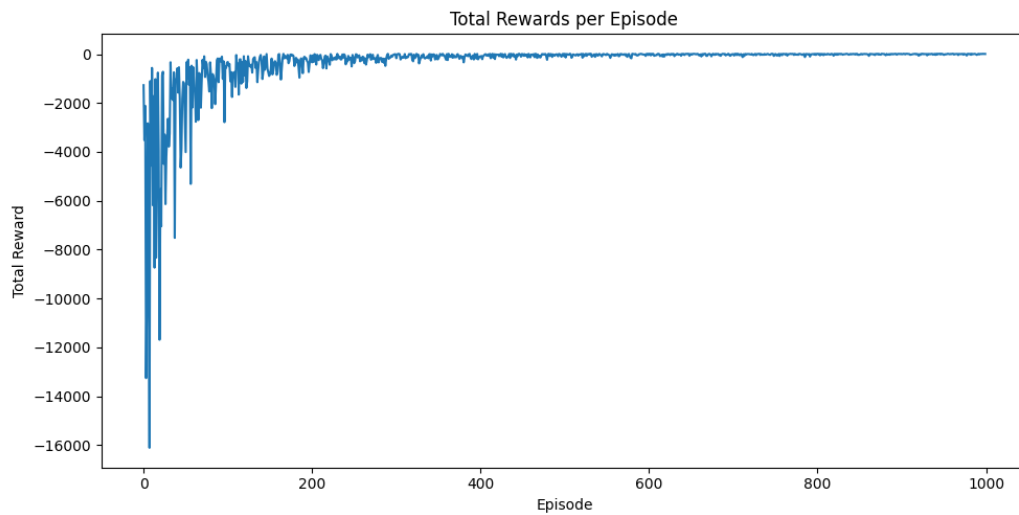
Experiment 5: High Discount Factor

- $\gamma = 0.99$ - Higher discount factor



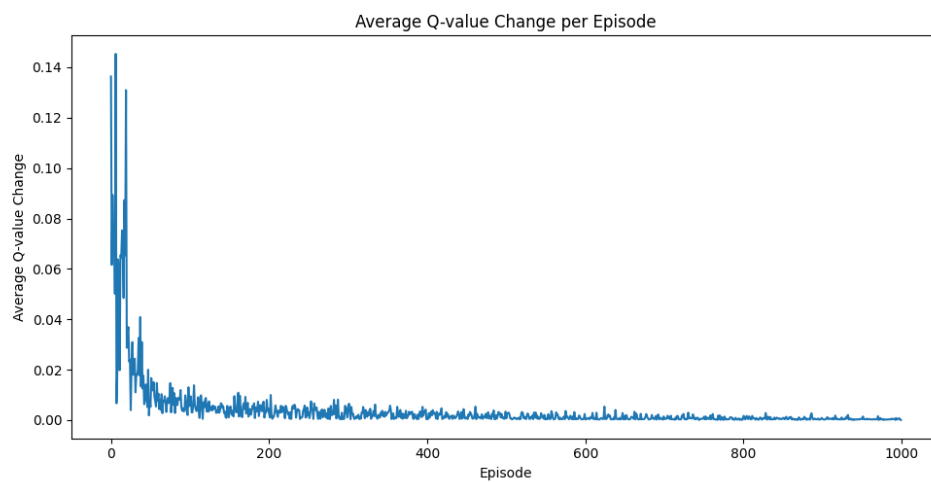
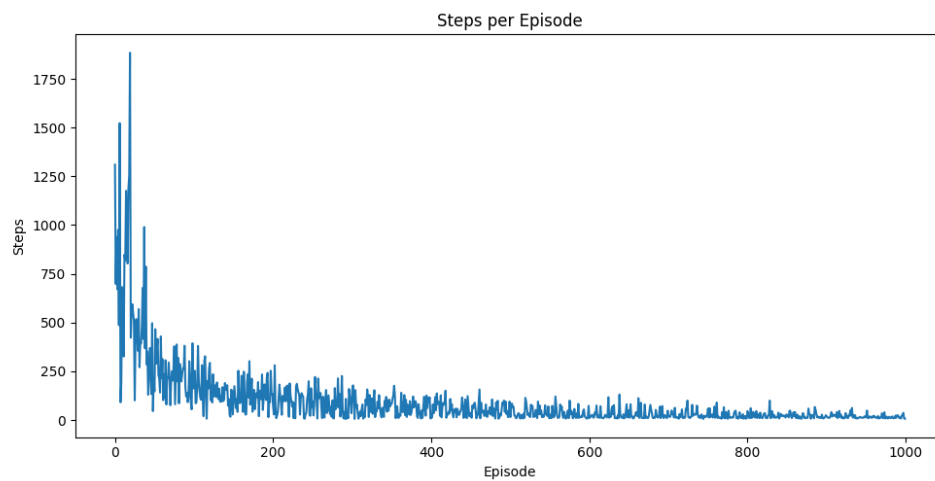
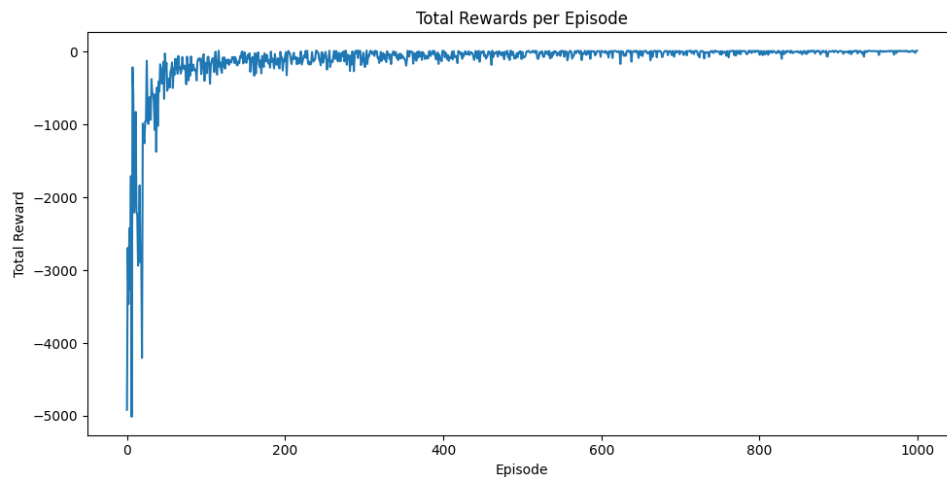
Experiment 6: Low Discount Factor

- $\gamma = 0.8$ - Lower discount factor



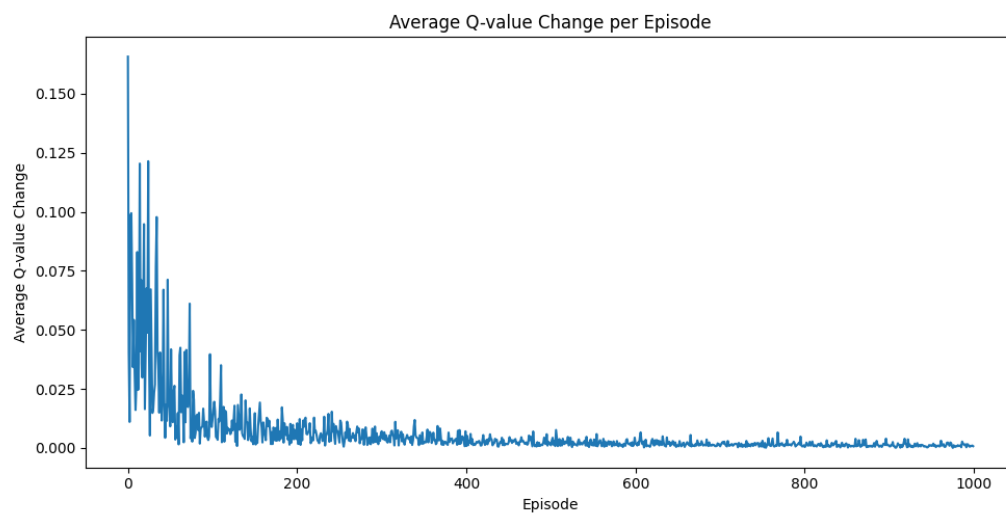
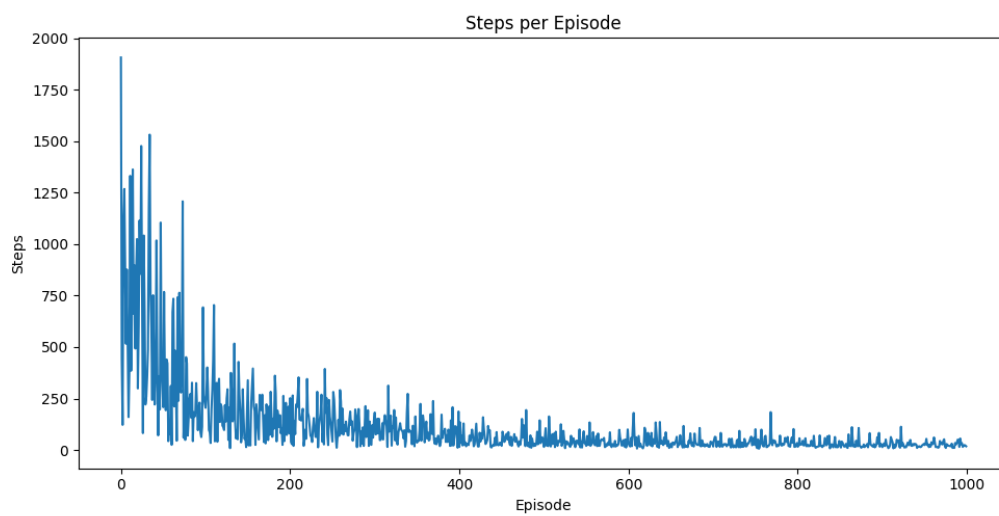
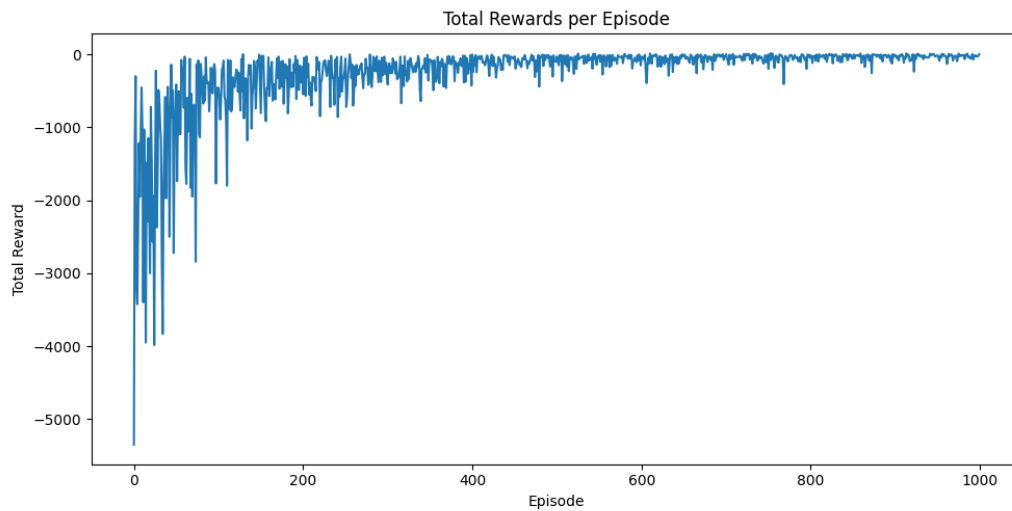
Experiment 7: High Initial Epsilon with Fast Decay

- $\text{epsilon_decay} = 0.99$ - Faster decay to reduce exploration quickly
- $\text{epsilon} = 1.0$ - Start with higher exploration



Experiment 8: Low Initial Epsilon with slow Decay

- $\epsilon = 0.5$ - Start with Lower exploration
- $\epsilon_{\text{decay}} = 0.9995$ - Slower decay to maintain a higher level of exploration longer



Observations:

- **Initialization:** Both zero and random initializations are effective, but zero initialization may lead to slightly more stable learning.
- **Learning Rate:** Higher learning rates can accelerate learning but may risk overshooting optimal policies. Lower rates are more stable but slower.
- **Discount Factor:** Higher discount factors encourage better long-term planning, leading to more optimal results.
- **Exploration Strategy:** Balancing exploration and exploitation through epsilon decay is crucial. Faster decay rates can lead to quicker but potentially less explorative learning.

Conclusions

The systematic experimentation with different hyperparameters and initialization strategies in the Taxi-v3 environment using Q-Learning has provided significant insights into the behavior and performance of the reinforcement learning agent. Here are the key conclusions drawn from the analysis:

Impact of Q-Table Initialization

- The experiments demonstrated that **initialization of the Q-table**, whether with all zeros or small random values, does not critically affect the long-term performance of the agent. Both methods allowed the agent to learn effectively, achieving similar rewards and efficiency by the end of the training. This suggests that the Q-Learning algorithm is robust to initial conditions in this specific task environment.

Influence of Learning Rate (Alpha)

- Adjusting the **learning rate** has a noticeable impact on the speed of learning. A **higher learning rate** allowed the agent to adapt quickly to the environment, evidenced by faster convergence in both total rewards and reduced steps per episode. However, it also introduced volatility in early learning stages. In contrast, a **lower learning rate** resulted in slower but more stable learning progress, indicating that it might be preferable when the agent operates in environments where stability and safety are priorities over rapid learning.

Effects of Discount Factor (Gamma)

- The **discount factor** significantly influences the agent's planning horizon. A **higher discount factor** encouraged the agent to value future rewards more, which led to more optimal long-term decisions, as seen in the higher stabilized rewards and more efficient navigation (fewer steps). A

lower discount factor made the agent focus more on immediate rewards, which might be useful in environments requiring quick, short-term gains but less effective for comprehensive task optimization.

Exploration Strategy (Epsilon Decay)

- The rate of **epsilon decay** affects the balance between exploration and exploitation. A **faster decay rate** quickly reduced the agent's exploration, promoting rapid exploitation of the learned policies. This was effective for achieving stability in simpler scenarios where optimal strategies can be learned quickly. Conversely, a **slower decay rate** maintained a higher level of exploration for longer, which could be beneficial in more complex or dynamic environments where continual adaptation to new states is necessary.

Overall Recommendations

Based on the findings, the following recommendations can be made for deploying Q-Learning in similar environments:

1. **Initialization:** Zero initialization is generally sufficient and simplifies the setup without compromising learning outcomes.
2. **Learning Rate:** Opt for a moderate to high learning rate if rapid learning is essential, but be cautious of potential instabilities. A lower rate is recommended when precision and safety are more critical.
3. **Discount Factor:** Use a higher discount factor to enhance long-term planning capabilities, especially beneficial in environments with complex goal structures.
4. **Exploration Strategy:** Adjust the epsilon decay based on the need for exploration. Faster decay is suitable for stable or known environments, while slower decay should be used in environments where the agent might encounter new, unseen scenarios frequently.